Chapter 1: Foundations of Algorithm Analysis

1.1 Asymptotic Notations and Big-O

Asymptotic analysis measures algorithm efficiency as input size n grows toward infinity. Big-O notation provides an asymptotic upper bound on worst-case execution time. Formally, f(n) = O(g(n)) if there exist positive constants c and n0 such that 0 <= f(n) <= c*g(n) for all n >= n0. Common complexity classes include O(1) constant time, O(log n) logarithmic, O(n) linear, O(n log n) linearithmic, and O(n^2) quadratic time.

Worst-case analysis guarantees the maximum running time for any input of size n. Average-case analysis computes expected execution time over a probability distribution of inputs. Best-case analysis identifies the fastest possible execution for favorable inputs.

1.2 Divide-and-Conquer Paradigm

The divide-and-conquer paradigm breaks a computational problem into smaller, independent subproblems of the same type, solves them recursively, and combines their solutions. MergeSort divides the input array of size n into two halves of size n/2, recursively sorts each half, and merges the two sorted sequences in O(n) linear time. The recurrence relation is T(n) = 2T(n/2) + O(n), yielding a guaranteed worst-case time complexity of O(n log n). However, standard MergeSort requires O(n) auxiliary memory space to perform the merge operation.

QuickSort selects a pivot element and partitions the array into elements smaller than and larger than the pivot. In the average case with random pivots, QuickSort executes in O(n log n) time and operates in-place, requiring only O(log n) auxiliary stack space for recursion. In the worst case—when the pivot is consistently the minimum or maximum element, such as on an already sorted array—the recurrence becomes T(n) = T(n-1) + O(n), resulting in O(n^2) quadratic running time.

1.3 Searching in Sorted Collections

Binary Search is an optimal search algorithm for sorted random-access sequences. At each step, the algorithm probes the median element at index mid = floor((low + high) / 2). If the target matches the median, search terminates successfully. If the target is smaller, the search continues in the left subarray; otherwise, it proceeds in the right subarray.

Because Binary Search eliminates half of the remaining candidate elements with each comparison, its recurrence relation is T(n) = T(n/2) + O(1). The maximum number of comparisons required for an array of size n is floor(log2(n)) + 1. For example, for an array with n = 1024 elements, Binary Search requires at most 10 comparisons, demonstrating exponential search-space reduction.

1.4 Dynamic Programming Foundations

Dynamic Programming (DP) is an algorithm design technique applied when a problem exhibits both optimal substructure and overlapping subproblems. Optimal substructure means that an optimal solution to the global problem contains optimal solutions to subproblems within it. Overlapping subproblems occur when the recursive space visits identical subproblems repeatedly.

Unlike Divide-and-Conquer which repeatedly resolves disjoint subproblems, Dynamic Programming stores the result of each solved subproblem in a lookup table. The two primary implementation strategies are top-down memoization (caching recursive call returns) and bottom-up tabulation (iteratively filling a table from base cases). A classic example is computing Fibonacci numbers: naive recursion requires O(2^n) exponential time, while dynamic programming achieves O(n) linear time with O(n) or O(1) space.